

Bypass Certificate Pinning

Privacy Analysis of Augmented Reality Glasses

1 Objective

The goal is to bypass the certificate pinning mechanism of the Meta View app on a phone running Android 14 to be able to use all functionalities like asking for the weather.

2 Additional Information

Note, that for the previous app and the Ray ban stories glasses, the app did not verify the certificate nor used certificate pinning. The bug reports were submitted as part of the BountyCon Events that were supported by the Meta Security Team. This vulnerability has since been fixed as we see when trying to capture traffic with mitmproxy. The following links to the website that explains the vulnerability in greater detail.

<https://code-byter.com/2023/09/24/rayban-hacking.html>

3 Methods

I tried different approaches, from modifying the APK to using runtime debugging and hooking frameworks. These efforts either tried to disable the certificate pinning checks or dynamically bypassing them during app execution.

4 Without Rooted Phone

4.1 Objection + Frida

Objection is a mobile exploration tool built on Frida, which supports bypassing certificate pinning dynamically without modifying the APK.

General steps:

1. Install Objection and Frida
2. Run the app on the phone and launch Frida-server on the phone

```
adb push frida-server /data/local/tmp/  
adb shell chmod 755 /data/local/tmp/frida-server  
adb shell /data/local/tmp/frida-server &
```

3. Explore the app using the command:

```
objection -g com.facebook.stella explore
```

4. Use the built-in command to disable certificate pinning:

```
android sslpinning disable
```

Problems:

- After running the objection explore command, it would indicate that it is using the phone but then it freezes, which prevents further interaction. Have to kill process to stop the exploration. This could be due to the restrictions on the phone.

- Repeated errors such as Address already in use occurred when attempting to start Frida. This was solved by verifying then killing the conflicting processes:

```
adb shell ps | grep frida
adb kill <processID>
```

- Errors encountered during Frida-server execution due to SELinux policies and non-rooted device restrictions. The error looks like:

```
Unable to load SELinux policy from the kernel: Failed to open file
?/sys/fs/selinux/policy?: Permission denied
```

4.2 APK Modification with Apktool

Decompile the APK, modify the code to disable certificate pinning, and recompile and resign the APK.

General steps:

1. Decompile the apk using apktool with the command:

```
apktool d -f base.apk -o base-decompiled
```

2. Search for methods related to certificate verifications, including 'checkServerTrusted', 'CertificatePinner', and 'HostnameVerifier' in the decompiled files.
3. Edit the methods to either return immediately or to bypass the certificate checks.
4. Rebuild the apk using the command:

```
apktool b base-decompiled -o base-rebuilt.apk
```

5. Sign the apk using the command:

```
java -jar uber-apk-signer.jar --apks rebuilt.apk
```

6. Install the edited apk on the phone:

```
adb install rebuilt.apk
```

Problems:

- There were errors due to invalid XML values in styles.xml and other hundreds of resource files. For example, the file would have:

```
<item name="APKTOOL_DUMMYVAL_0x7f040043">false</item>
```

I tried to solve this by replacing them with mostly 'null' or proper values.

- There were errors when compiling the resources. The error looked like:

```
invalid resource directory name: /res navigation
```

Note that this directory does not exist for me, so I was not able to solve this compiling issue. I tried to avoid the decompilation of the resource entirely by using the '-no-res' flag, but that did not work.

- apk did not rebuild properly. I tried using '-use-aapt2' to attempt rebuilding the apk but faced additional errors related to resource compilation and malformed XML files.

4.3 Runtime Hooking with Frida

Hook into runtime methods like 'checkServerTrusted' to bypass certificate pinning during execution.

General steps:

1. Write a custom Frida script to hook into the method.
2. Inject the script using the command:

```
frida -U -n com.facebook.stella -e <script.js>
```

Problems:

- Non-rooted device restrictions prevented effective hooking.
- Frida-server issues due to SELinux and app protection mechanisms. The error looks like:

```
Unable to load SELinux policy from the kernel: Failed to open file
?/sys/fs/selinux/policy?: Permission denied
```

4.4 APK Patching with apk-mitm

apk-mitm is a tool that automatically removes certificate pinning from Android APK files.

General steps:

1. Install apk-mitm
2. Run apk-mitm which does general SSL pinning bypass by applying standard patches to the apk using the command:

```
apk-mitm base.apk
```

Or, to specify the mitmproxy certificate into the app's network security configuration in addition to patching SSL pinning, meaning to use apk-mitm to modify the app and inject the mitmproxy certificate:

```
apk-mitm --certificate ~/Downloads/mitmproxy.pem ~/Downloads/base.apk
```

Problems:

- The building of the resources fails due to malformed XML files. This is how it should look:

```
$ apk-mitm example.apk
✓ Decoding APK file
✓ Modifying app manifest
✓ Replacing network security config
✓ Disabling certificate pinning
✓ Encoding patched APK file
✓ Signing patched APK file
```

And this is what I see:

```
✓ Checking prerequisites
✓ Decoding APK file
✓ Applying patches
} Encoding patched APK file
↓ Encoding using AAPT2 [skipped]
→ Failed, falling back to AAPT...
✓ Encoding using AAPT [fallback]
→ I: Building resources...
  Signing patched APK file
...
Failed! An error occurred:
```

```

...
I:Building resources
...
W: <tmp_dir>/decode/res/layout/APKTOOL_DUMMYVAL_0x7f0e0025.xml:7:
error: not well-formed (invalid token).
...

```

4.5 Android SSL Unpinning

The Android SSL modifies the apk to disable SSL pinning dynamically by injecting Frida and HTTP Toolkit's unpinning scripts. General steps:

1. Install android unpinner
2. Run the command:

```
android-unpinner all base.apk
```

Problems:

- The process aborted. The following is what I see:

```

[18:33:59] Target: com.facebook.stella
[18:33:59] Reusing existing file: /home/michelle/Downloads/base.unpinned.apk
[18:33:59] Installing com.facebook.stella...
[18:34:07] Detect architecture...
[18:34:07] Copying matching gadget: frida-gadget-16.0.10-android-arm64.so...
[18:34:08] Copying builtin Frida scripts to /data/local/tmp/android-unpinner...
[18:34:08] Active frida scripts: ['hide-debugger.js', 'httptoolkit-unpinner.js']
[18:34:08] Start app (suspended)...
[18:34:08] Obtain process id...
[18:34:10] Establish Java Debug Wire Protocol Connection over ADB...
Aborted.

```

The issue appears to stem from JDWP connection which could be due to the phone not allowing adb debugging for the app.

5 With Rooted Phone

The initial setup with mitmproxy shows that although mitmproxy was able to intercept unencrypted HTTP traffic but failed to decrypt HTTPS traffic, and apps displayed TLS handshake errors. This is due to SSL pinning. The following is an example log:

```

Client TLS handshake failed. The client does not trust the proxy's
certificate for graph.instagram.com (OpenSSL Error([('SSL routines',
'', 'ssl3 alert certificate unknown'])))

```

The reason for this is because like most monder apps, Meta View uses SSL pinning, rejecting user-installed certificates.

5.1 Objection + Frida

General Steps:

1. Push and start the Frida server on the phone using the following commands (change the frida-server to what was downloaded):

```
adb push frida-server /data/local/tmp/
adb shell
su
setenforce 0
chmod +x /data/local/tmp/frida-server
/data/local/tmp/frida-server &
```

2. Verify that the Frida server is running:

```
adb shell ps | grep frida
```

3. Use Objection to disable SSL pinning:

```
objection -g com.facebook.stella explore
android sslpinning disable
```

Problems:

- The result shows that Objection appeared to disable SSL pinning, but mitmproxy still encountered TLS handshake errors for certain domains (e.g., graph.instagram.com). The reason is most likely due to the app using additional SSL checks or custom implementations not covered by Objection.

5.2 Custom Frida Scripts

General Steps:

1. Decompile the APK.
2. Locate methods such as **checkServerTrusted** in smali files.
3. Push and start Frida server on the phone:

```
adb push frida-server /data/local/tmp/
adb shell "chmod 755 /data/local/tmp/frida-server"
adb shell "/data/local/tmp/frida-server &"
```

4. Write a simple custom Frida script to hook into these methods and bypass SSL pinning:

```
Java.perform(function () {
    var TrustManagerImpl = Java.use("javax.net.ssl.TrustManagerImpl");
    TrustManagerImpl.checkServerTrusted.implementation = function (chain,
    authType) {
        console.log("Bypassing TrustManagerImpl");
    };
});
```

5. Run the script using Frida with the following command (but change com.facebook.stella into the name of the package for the app and change disable_sslpinning.js to the name of the json file for the custom script):

```
frida -U -f com.facebook.stella -l disable_sslpinning.js
```

Problems:

- The app crashes during runtime. This could either be due to obfuscation in the app preventing the hooks from being correctly applied and/or that process and code of the dynamic hooking was not implemented correctly. The error logs look like:

```
TypeError: cannot read property 'overload' of undefined
Process crashed: java.lang.IllegalStateException:
AppContext.set has not been invoked
```

5.3 Transparent Proxy Mode

General Steps:

1. Enable transparent mode when starting mitmproxy using the following command:

```
mitmproxy --mode transparent --showhost
```

2. Configure iptables to redirect traffic to mitmproxy

```
adb shell
su
iptables -t nat -A OUTPUT -p tcp --dport 443 -j REDIRECT --to-port 8080
```

3. Verify iptables rules:

```
adb shell iptables -t nat -L -n -v
```

Problems:

- mitmproxy was able to intercept general HTTPS traffic (e.g., browser traffic) however the traffic of the app was blocked. The reason could be because the app uses custom SSL pinning logic or additional protections beyond standard pinning.

5.4 Installing Certificate as a System Certificate

General Steps:

1. Connect your phone via USB and open a terminal and write:

```
adb shell
su
setenforce 0
```

2. Ensure the mitmproxy certificate is in .pem format. If it's already on the phone, rename it to **mitmproxy-ca-cert.pem**. If it's on the laptop, then transfer it to the phone using:

```
adb push /path/to/mitmproxy-ca-cert.pem /sdcard/Download/
```

3. Get the hash value of the certificate on the phone(The certificate was in /sdcard/Download for me):

```
openssl x509 -inform PEM -subject_hash_old -in /sdcard/Download/
mitmproxy-ca-cert.pem | head -1
```

4. Either rename the certificate on the phone

```
mv /sdcard/Download/mitmproxy-ca-cert.pem /sdcard/Download/<hash>.0
```

5. Or rename the certificate on the laptop to its hash value then copy the certificate to the system certificate store using the commands using the commands:

```
mv mitmproxy-ca-cert.pem <hash>.0
adb shell
su
mount -o rw,remount /system
cp /sdcard/Download/<hash>.0 /system/etc/security/cacerts/
chmod 644 /system/etc/security/cacerts/<hash>.0
```

6. Reboot the phone

Problems:

- The mounting of the system partition as writable did not work. The reason for this is due to Android 14's enhanced security measures, particularly around the /system partition. It looks like:

```
moonstone:/ # mount -o rw,remount /system
mount: '/system' not in /proc/mounts
1|moonstone:/ # mount -o rw,remount /
'/dev/block/dm-6' is read-only
```

5.5 Magisk

5.5.1 Direct Installation vis NVISO trustusercerts

General Steps:

1. Connect your phone via USB and open a terminal and write:

```
adb shell
su
setenforce 0
```

2. Ensure the mitmproxy certificate is in .pem format. If it's already on the phone, rename it to mitmproxy-ca-cert.pem. If it's on the laptop, then transfer it to the phone using:

```
adb push /path/to/mitmproxy-ca-cert.pem /sdcard/Download/
```

3. Get the hash value of the certificate on the phone(The certificate was in /sdcard/Download for me):

```
openssl x509 -inform PEM -subject_hash_old -in /sdcard/Download/
mitmproxy-ca-cert.pem | head -1
```

4. Rename the certificate

```
mv /sdcard/Download/mitmproxy-ca-cert.pem /sdcard/Download/<hash>.0
```

5. Install the zip file of NVISO **always trustusercerts** from the github repository
<https://github.com/NVISOsecurity/MagiskTrustUserCerts>.

6. Install the NVISO **always trustusercerts** Magisk Module in Magisk.

7. Reboot the phone.

8. Verify that the module is active on Magisk.

9. Copy the mitmproxy certificate from the Downloads folder to Magisk module directory.

```
cp /sdcard/Download/<hash>.0 /data/adb/modules/trustusercerts/
system/etc/security/cacerts/
```

10. Reboot the phone

11. Verify if the certificate is in the **cacerts** directory and is applied to the system's certificate store:

```
ls -l /data/adb/modules/trustusercerts/system/etc/security/cacerts/
ls -l /system/etc/security/cacerts/
```

Problems:

- The certificate did not appear in the system certificate and the app still failed to trust mitmproxy's certificate. The reason could be due to SELinux policies and potential issues with the Magisk module prevented the changes from taking effect.

5.5.2 Direct Installation vis NVISO trustusercerts

General Steps:

1. Create a custom Magisk module on the laptop using the commands:

```
mkdir -p mitmproxy_module/system/etc/security/cacerts
mkdir -p mitmproxy_module/META-INF/com/google/android
```

2. Make sure to have the certificate with the hashed name on the laptop.

3. The structure of the modules should look like:

```

mitmproxy_module/
  META-INF/
    com/
      google/
        android/
          update-binary
          updater-script
  post-fs-data.sh
  module.prop
  system/
    etc/
      security/
        cacerts/
          <hash>.0

```

4. To create a file, use the command:

```
nano mitmproxy_module/module.prop
```

The content of the files should be:

```

module.prop:
id=mitmproxycert
name=MITM Proxy Certificate
version=1.0
versionCode=1
author=Michelle
description=Install mitmproxy certificate as system-level trusted.

post-fs-data.sh:
#!/system/bin/sh
MODDIR=${0%/*}

mkdir -p $MODDIR/system/etc/security/cacerts
rm $MODDIR/system/etc/security/cacerts/*
cp -f /data/misc/user/0/cacerts-added/* $MODDIR/system/etc/security/cacerts/

update-binary:
#!/sbin/sh
umask 022
OUTFD=$2
ZIPFILE=$3

mount /data 2>/dev/null

[ -f /data/adb/magisk/util_functions.sh ] || exit 1
. /data/adb/magisk/util_functions.sh
install_module
exit 0

updater-script:
#MAGISK

```

5. Verify the permissions for all files:

```

chmod -R 644 mitmproxy_module/*
chmod +x mitmproxy_module/post-fs-data.sh
chmod +x mitmproxy_module/META-INF/com/google/android/update-binary

```

6. Zip the module in the directory of the mitmproxymodule, if there is any permission issue try running zip with sudo:

```
cd mitmproxy_module
zip -r ../mitmproxy_module.zip ./*
```

7. Verify the ZIP structure using:

```
unzip -l mitmproxy_module.zip
```

The output should look like:

```
META-INF/com/google/android/update-binary
META-INF/com/google/android/updater-script
module.prop
post-fs-data.sh
system/etc/security/cacerts/<hashed-cert>.0
```

8. Transfer the module to the phone:

```
adb push mitmproxy_module.zip /sdcard/Download/
```

9. Go to the Magisk app and **Modules**.
10. Tap on **Install from Storage** to install the module via Magisk.
11. Reboot the phone
12. Verify that the certificate is moved to the system certificate:

```
ls -l /system/etc/security/cacerts/
```

Problems:

- Permission denied when trying to modify the file. The error looks like:

```
Error writing mitmproxy_module/post-fs-data.sh: Permission denied
```

Resolve it by first ensuring you have ownership of the mitmproxy_module directory and all its files:

```
sudo chown -R $(whoami):$(whoami) mitmproxy_module/
```

Then set write permissions:

```
chmod +w mitmproxy_module/post-fs-data.sh
```

- Magisk failed to install the module